

Realidad Virtual

Entorno inmersivo para estímulo del desarrollo infantil

MARTÍN ALBARRACÍN, Javier Andrés

Universidad Nacional de Cuyo - Facultad de Ingeniería

Cátedra: Realidad Virtual

Docentes: ROSENSTEIN, Javier - TOMBA, Carlos

Abril 2026

1 MOTIVACIÓN

Las herramientas de realidad virtual tienen gran variedad de aplicaciones de uso, tanto en el plano educativo o instructivo como en el plano recreativo. El proceso de estímulo para el desarrollo infantil tiene justamente estos dos componentes: aprendizaje y juego.

En la última década, han crecido significativamente los trastornos de neurodesarrollo, tanto en el país como en el mundo. Por un lado, hay una creciente cantidad de niños en esta condición, pero por otro lado hay mayor y mejor diagnóstico de estos trastornos, junto con tratamientos tempranos que ayudan al desarrollo normal del niño. La propuesta es generar una herramienta complementaria a las tradicionales para este tipo de terapias.

En ese marco, el objetivo de este proyecto fue crear un entorno inmersivo, cuyos objetos virtuales sean de interés para el niño que está inmerso en el juego, y con los cuales deba interactuar para descubrir lo que hacen, de manera tal que se estimule el habla, equilibrio y motricidad.

2 RESUMEN

Se trabaja la inmersión con lentes de realidad virtual Meta Quest 3s, con una vara que aloja un microcontrolador Node MCU ESP 8266, que maneja un módulo de reconocimiento de voz y un giróscopo, con los cuales envía información para el procesamiento al Meta Quest. El proyecto es híbrido: comienza con un escenario de realidad mixta donde hay tres portales, cada uno de ellos traslada al usuario a un mundo virtual diferente, donde cada uno tiene una utilidad específica:

- Granja Virtual: en este escenario se pretende estimular el habla, en un escenario en el cual se puede crear un animal al agitar la vara nombrándolo. La escena tiene 5 asistentes, cada uno de los cuales muestra como crear un animal usando la vara cuando el usuario se acerca.
- Ciudad Virtual: se trata de un escenario de una ciudad futurista. En este escenario, hay una serie de plataformas virtuales móviles, para que el niño salte de una a otra y se desplace por la ciudad virtual en el aire. Inicialmente, las plataformas están ocultas, y aparecen al levantar la vara por encima de la cabeza. Este escenario ayuda al niño a trabajar su equilibrio
- Isla Virtual: se trata de una ruta ubicada junto a un lago con una isla. Al ingresar en este entorno, hay un auto volador estacionado junto a un tablero con figuras geométricas. El usuario se sube al auto y cuando dibuja las figuras con la vara, el auto arranca y da un paseo sobrevolando la isla. La idea de este escenario es estimular la motricidad fina.

3 COMPONENTES DEL PROYECTO

El proyecto se realizó en dos versiones: una versión completa, que es la descrita en el resumen, y una versión simplificada, pensada para el uso sin la vara, lo cual requiere algunas adaptaciones que se describirán en la sección correspondiente.

3.1 Versión Completa

En el esquema de la Imagen 1 puede verse la estructura general del proyecto, en el que hay:

- Lentes de RV que permitan realidad mixta. Aquí corre la aplicación principal, que se comunica con los demás componentes y tiene habilitada la función handtracking para virtualizar la posición de las manos.
- La vara, que está comandada por un microcontrolador Node MCU ESP8266, con conectividad WiFi, que se comunica con los dos sensores: un módulo de reconocimiento de voz VoiceRecognition Elechouse y un giróscopo MPU 6500. Sobre este microcontrolador se monta el server y los demás componentes se conectan vía WebSocket al mismo como clientes.
- Un archivo `<html>` que puede conectarse como cliente al servidor. Este sirve como bypass al reconocimiento de voz, de modo que un operador externo puede indicar que el usuario en juego ha dicho una determinada palabra, aunque no la reconozca el módulo de reconocimiento de voz. Este archivo `<html>` se llama intérprete humano de voz, y puede ser ejecutado desde cualquier plataforma o dispositivo con un navegador web, lo que le da gran versatilidad.

Cabe destacar que tanto el Intérprete Humano de Voz como los lentes de RV se conectan al mismo servidor, es decir que "escuchan" a través de una IP fija. Para el caso, se ha utilizado como red un extensor TP-Link con sus propias credenciales de acceso, que retransmite la señal WiFi.

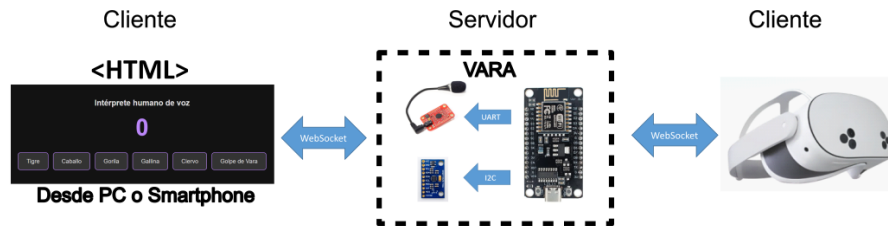


Imagen 1: Esquema de componentes

3.2 Versión Simplificada

Esta versión del proyecto está orientada al uso de los lentes de RV con los mandos (sin la vara, o simulando la vara con el mando derecho). En este contexto, el Intérprete Humano de Voz se ha reconfigurado completamente, de modo que se tiene una GUI similar a la anterior, pero en este caso se trata de una aplicación realizada en python donde se monta el servidor, ya que en ese caso no hay un Node MCU que cree el server. Por su parte, el Meta Quest se comunica como cliente, y utiliza los mandos en lugar del handtracking.

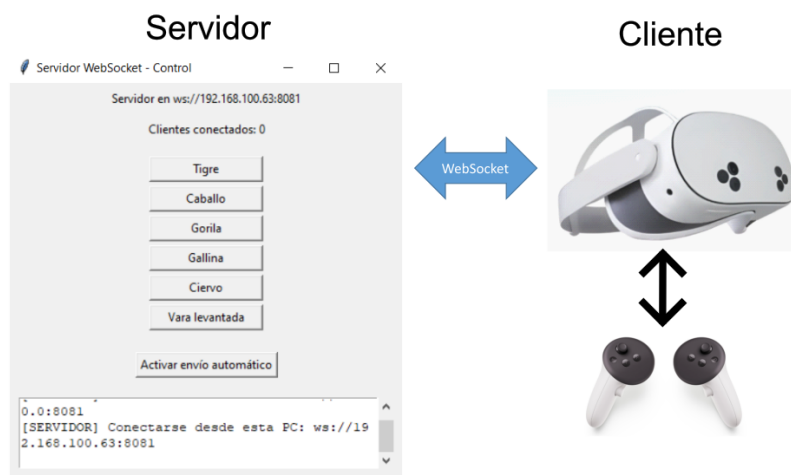


Imagen 2: Esquema de Componentes

Vale la pena mencionar que existe más de una forma de conectar hardware no nativo con Meta Quest. En este caso se valoró la utilización de bluetooth y Web Socket. Finalmente, se optó por Web Socket debido a que este tipo de comunicación tiene más baja latencia y se puede usar de forma directa, sin necesidad de emparejar previamente los dispositivos; a pesar de que su implementación es más compleja.

4 CONSIDERACIONES DEL HARDWARE

En esta sección se detallará el hardware utilizado en la versión completa, mencionando qué componentes no son necesarios en la versión simplificada:

1. Meta Quest 3s: sobre estos lentes de RV corre la aplicación, que es la versión más económica del modelo con realidad mixta, sin embargo el proeyecto es compatible con Meta Quest 3 y Meta Quest Pro. Dado que el inicio de la app es un escenario de realidad aumentada, no es compatible Meta Quest 2, que no dispone de pass through. No obstante, sería un trabajo muy sencillo eliminar la realidad mixta y adaptar el entorno inicial a realidad virtual 100%. Cabe destacar que para el desarrollo de esta aplicación en Unity es necesario contar con una placa aceleradora gráfica, en este caso se dispone de la placa Nvidia 3060 y 16Gb de memoria RAM. Para la versión completa del entorno, no es necesario el uso de los mandos (de hecho, es recomendable no usarlos para no interferir con el handtracking) mientras que para la versión simplificada se recomienda el uso de los mandos.



Imagen 3: Lentes de realidad virtual y sus mandos

2. Hardware de la vara: la vara consta del micro-controlador conectado a los sensores de reconocimiento de voz y giroscopio. Los aspectos principales de estos componentes son los siguientes:
 - ESP-8266: funciona con 3,3V pero puede alimentarse con 5V regu-

lados y estables. En este caso, la alimentación se realizó mediante una Power Bank. Ambos sensores pueden trabajar correctamente con este nivel de tensión.

- **Giróscopo y acelerómetro MPU6500:** permite medir aceleración, incluyendo en el eje vertical la aceleración de la gravedad. Cabe destacar que la aceleración de la gravedad se distribuye en componentes de cada eje (x, y, z) según la orientación del giroscopio, de modo que su composición da g cuando el sensor está quieto. Se obtienen tres valores: (ax, ay, az) que son las componentes según la orientación del sensor. Este sensor también tiene un giroscopio, que permite obtener la terna de valores de velocidad de rotación del sensor (gx, gy, gz) . Estos valores se procesan dentro del micro-controlador luego de sensarlos, a fin de obtener dos valores lógicos: el primero indica si la vara se ha girado por encima de cierta velocidad (acción de agitar la vara) y el segundo indica si la vara se está desplazando a cierta velocidad total (para detectar si el usuario está elevando la vara). La toma de los valores del sensor se realiza mediante comunicación I2C.
- **Módulo Voice Recognizer:** este dispositivo tiene 80 canales en los cuales se pueden entrenar sonidos para reconocerlos posteriormente. No obstante, el módulo puede cargar hasta siete sonidos de los que tiene pre-entrenados durante su funcionamiento normal, por lo que se limita en términos prácticos a siete valores reconocibles. La toma de estos valores se realiza por protocolo UART. Cabe destacar que, si bien el módulo funciona correctamente, no resulta el dispositivo más apropiado para este fin, ya que la voz que reconoce debe ser muy parecido a la voz con la que se entrenó, de modo que no es sensible a los cambios de tono y timbre de voz, por lo que en la práctica se optó por puentear este reconocimiento de voz, mediante el intérprete de voz humano.

En la siguiente imagen puede verse un esquemático de las conexiones:

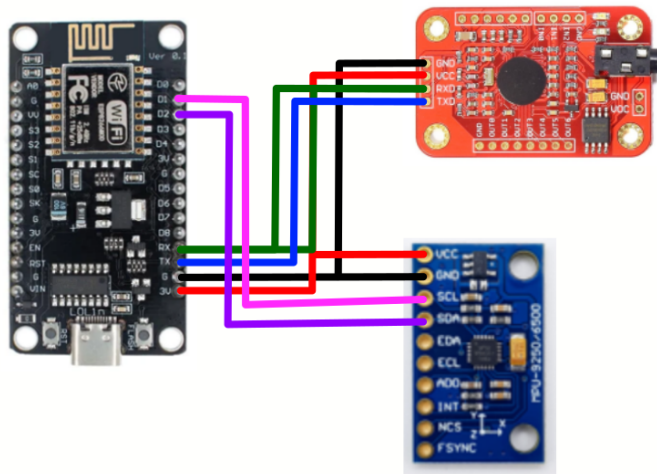


Imagen 4: Conexión física de componentes de la vara

3. Extensor de señal WiFi TP-Link: Para el correcto funcionamiento de la comunicación entre el micro-controlador y los lentes RV, ambos deben estar conectados a la misma red WiFi, no obstante, no se necesita acceso a internet. El servidor ocupa una determinada dirección IP y un puerto predefinido, los clientes escuchan ese puerto. Dado que se va a usar una IP fija, se considera adecuado el uso de una red dedicada, en la cual se asegura la disponibilidad de esa IP y ese puerto. Para esto, se ocupa un extensor de señal como el mostrado a continuación:



Imagen 5: Extensor de señal WiFi

Cabe destacar que, si se utilizara la red WiFi local, habría que reconfigurar la aplicación para utilizar tanto la IP como las credenciales de acceso correctas. En cambio, el uso de este dispositivo permite utilizar siempre la misma credencial de acceso, aunque no tenga acceso a internet, simplemente conectando a la red eléctrica el extensor de señal.

5 CONSIDERACIONES DE SOFTWARE

A continuación se describirá y mostrará como está estructurado el software del proyecto, desarrollado en Unity y sus componentes adicionales (Intérprete de voz y vara). Se describirán también las variantes implementadas para la versión simplificada. En esta descripción no se mostrarán los programas completos y la explicación de sus códigos, sino más bien una descripción del funcionamiento, y podrá accederse al código fuente en el repositorio referenciado al final de este informe, junto con material audiovisual complementario.

5.1 Versión Completa

Unity trabaja con objetos de escena que pueden tener objetos hijos en su jerarquía. Cada uno de estos objetos puede estar activo o inactivo, y puede ejecutar un script durante su vida en la escena. Los objetos también pueden tener otros atributos, entre los cuales destacan dos: el collider (en su forma más simple es una caja que circunscribe al objeto) y el rigidbody, necesarios para la detección de colisiones.

5.1.1 Escenario inicial

- Avatar del Personaje Se ha utilizado un avatar de mixamo para el personaje, con full-body tracking nativo de Meta Quest (sin) para situar el avatar en la escena . El mismo se sitúa en la posición inicial de la escena, y está conformado por un GameObject con los siguientes elementos hijos en su jerarquía:



Imagen 6: Enter Caption

- Prefab del modelo: se configura como Humanoid. Dentro de este objeto hay jerarquía compleja de objetos que representa el cuerpo desde las caderas hasta los dedos. Dentro de esta jerarquía se crea la vara como objeto hijo de la mano derecha. Hay tres varas, las cuales empiezan todas inactivas, y se activará la correspondiente al mundo virtual en el que el usuario entre.
- Building block del passthrough, para que al inicio se pueda ver el entorno físico real (en realidad mixta).
- Building block para space tracking (uso de los controles mandos y cabezal)
- Building block para full body tracking: utiliza las cámaras de meta quest y la posición y orientación del cabezal y los mandos para determinar la posición corporal del jugador.

Así mismo el avatar (objeto padre) tiene scripts que se importan con los building blocks para controlar su movimiento. Para gestionar colisiones, se crea un box-collider y un rigidoby, el cual debe tener activa la opción isKinematica e inactiva gravity, para moverse completamente en forma cinemática, según los scripts que están corriendo asociados al avatar. Además de los scripts propios de los building blocks usados, se creó un script propio para el personaje, llamado "PlatformParent", que tiene la siguiente funcionalidad:

- Cuando se sitúa justo encima de un objeto del tipo "MovingPlatform" se hace hijo de este, lo cual hará que se mueva y rote junto con él. Esto se usará en la ciudad virtual y en la isla virtual.
- Cuando se sitúa encima de un objeto cualquiera, queda apoyado y no se mueve verticalmente. En caso contrario se aplica una "gravedad artificial" por código (no por la física nativa de Unity) que hace que caiga cinemáticamente con una aceleración menor a la aceleración de la gravedad, para evitar la sensación de vértigo.
- Cuando está cayendo, se detecta periódicamente la distancia hacia abajo, y cuando está cerca del suelo, desciende su velocidad hasta un valor bajo (no nulo) con el que llegará al suelo. Esto se hace para evitar caer a una velocidad muy alta, que genera un golpe molesto sobre el ojo al parar repentinamente la escena en el contacto con el piso del avatar.
- La detección del objeto debajo del avatar se realiza mediante raycast, disponible para Unity en C#.

- Escena y portales

La escena original tiene habilitado el passthrough, que permite ver el entorno que captan las cámaras del metaquest, para trabajar con realidad mixta. En este escenario hay tres objetos 3D que son los portales, cada

uno de los cuales lleva a uno de los mundos virtuales. Cabe destacar que para estos objetos, como para la mayoría los objetos descargados desde assetsUnity, es necesario reconfigurar los materiales como URP después de importar el unitypackage. En el repositorio referenciado al final de este informe, se han subido el proyecto comprimido en .rar a fin de evitar esta tarea al momento de descargarlo. Los escenarios virtuales, al inicio del juego están desactivados.

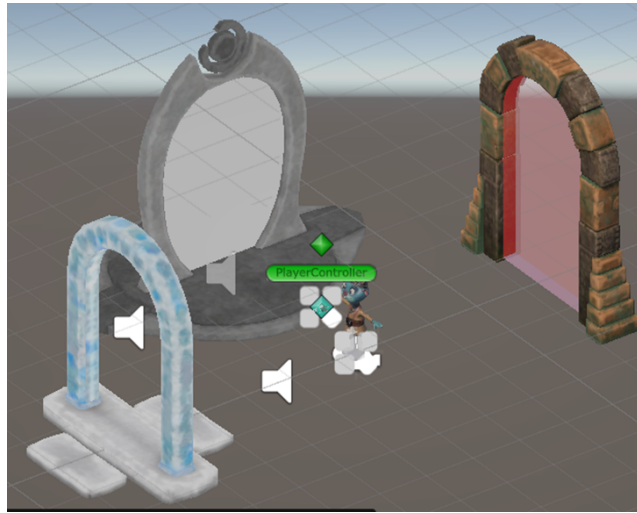


Imagen 7: Enter Caption

- Interfaz portal-escenarios virtuales En el portal, se ha colocado un cubo transparente con un boxcollider y un script llamado "PortalPositionTrigger" que realiza la siguiente funcionalidad:
 - Detecta si el portal está entrando en colisión con el Player, en cuyo caso deshabilita el passthrough, habilita la vara correspondiente al mundo en el que va a entrar y habilita el propio mundo. También reproduce un audio de bienvenida al escenario virtual, y deshabilita los otros dos portales.
 - Cuando atraviesa el portal desde adentro hacia afuera, hace la tarea contraria: inhabilita la vara y el mundo virtual, al mismo tiempo que vuelve a habilitar el passthrough para salir de la realidad virtual y volver a la realidad mixta.
- Cliente WebSocket: Hay un objeto padre llamado WebSocket, que implementa el cliente que escucha el puerto (en la IP asignada) por el cual el servidor establece la comunicación. A través de este puerto llega la información codificada en un número, el cual se le pasa a uno de los dos objetos hijos que tiene, lo que puede representar: crear un animal con la

vara en la granja o mostrar el camino en la ciudad virtual. Cada uno de estos objetos hijos tiene su propio script, cuya funcionalidad se abordará en la sección de cada escenario.

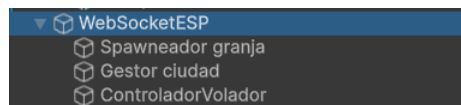


Imagen 8: Jerarquía del GameObject para el WebSocket

5.1.2 Granja Virtual

- Entorno (ambiente) El entorno se ha descargado del sitio oficial de Unity, consiste en un escenario donde hay colinas verdes, árboles y vegetación. El escenario es bastante extenso, el usuario podría caminar mucho a través de él, pero se ha restringido el espacio de utilidad a una extensión mucho menor.

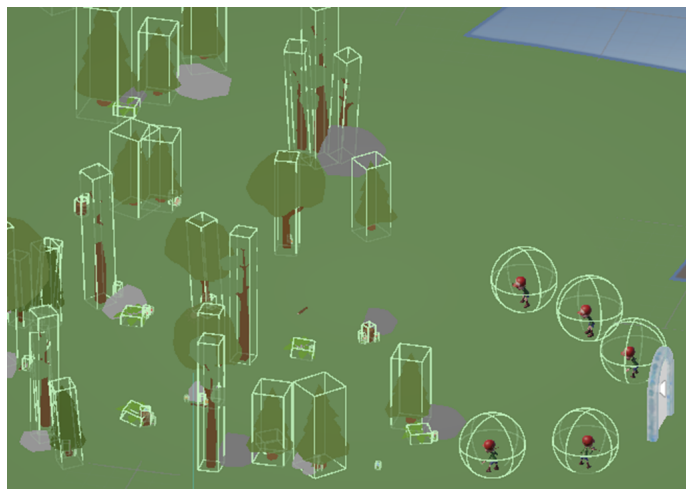


Imagen 9: Vista de la granja virtual

- Asistentes de la Granja Hay cinco asistentes de granja, que son personajes de Mixamo igual que el personaje, en este caso todos iguales. El asistente tiene un prefab asignado para crear un animal y un audio para decir en voz alta el animal que crea. La dinámica del asistente es la siguiente: tiene un sphere-collider, que dispara pasa de la animación por defecto "quieto" a la animación "agitarVara" cuando el jugador se acerca. Esto se realiza llamando al método SpawnAnimal del script ColiderAsistenteGranja, que se habilita al entrar en contacto con el jugador.

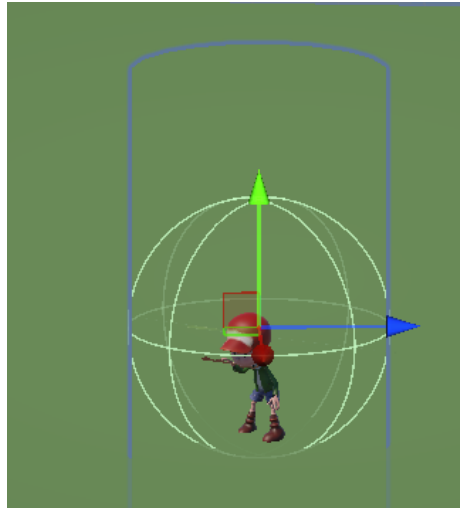


Imagen 10: Asistente de la granja y zona de contacto

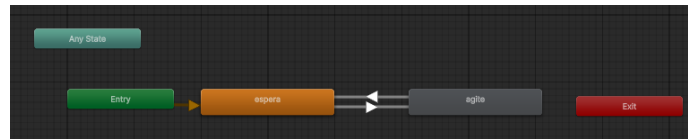


Imagen 11: Animator del asistente de la granja

- Interacción del usuario El usuario interactúa con el entorno a través de la vara, desde el WebSocket server llega la información que provee la vara con la detección de voz y el movimiento de la misma, sintetizada en los siguientes comandos numéricos útiles para esta escena:
 - 1: Crear tigre
 - 2: Crear caballo
 - 3: Crear gorila
 - 4: Crear gallina
 - 5: Crear ciervo
- Prefabs y creación de animales

Para cada uno de estos animales, se ha creado un prefab (objeto prefabricado) que aparece en escena cuando llega el comando numérico que corresponde (esto ocurre cuando el jugador agita la vara y dice el nombre del animal). El prefab tiene una animación consistente en una traslación y escalado progresivo (aparece

diminuto y llega a su tamaño normal) por única vez, seguido de una animación nativa del animal descargado, que se realiza en bucle.



Imagen 12: Ejemplo de animales en la granja

Cabe destacar que cuando el animal aparece en escena, el script lo hace hijo del escenario virtual, a fin de que al salir de la granja por el portal, todos los animales creados desaparezcan junto con la granja. Se ha definido como punto de creación de animales un punto en el extremo de la vara, con la orientación del avatar al momento de crear el animal.

5.1.3 Ciudad Virtual

- Entorno (ambiente) Este entorno también es bastante extenso, y no se han incorporado animaciones a sus objetos, que son varios (autos, autos voladores, carteles, edificios, etc...). No se prevee que el usuario camine a través del escenario, sino que se desplace a través de las plataformas voladoras.

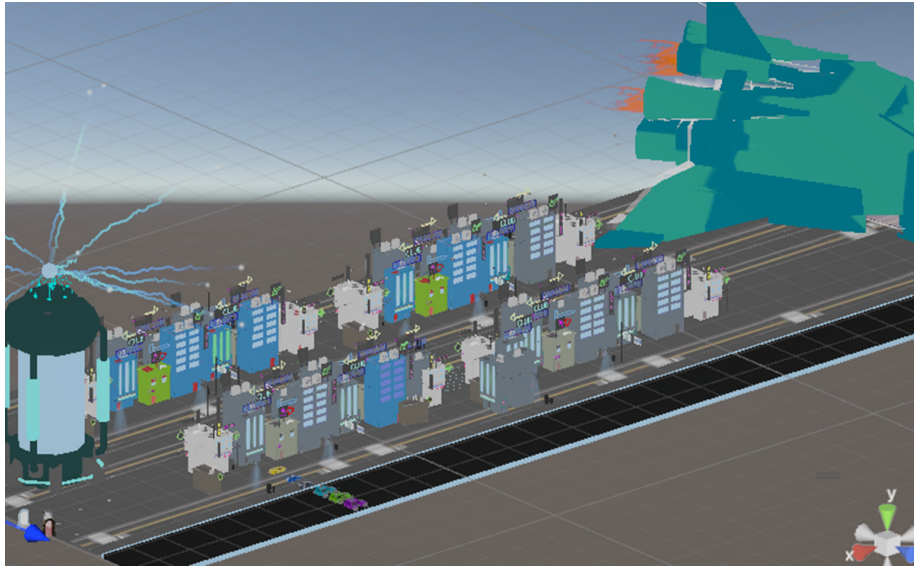


Imagen 13: Vista de la ciudad virtual

- Plataformas voladoras y scripts de la escena: Las plataformas son prefabs que consisten en un disco de superficie un poco mayor a la que ocupa el avatar, que tienen el tag "MovingPlatform" para que el avatar del personaje pueda adherirse a él. Además, cuentan con el boxcollider, rigidbody y los siguientes scripts:
 - Tres scripts para movimientos lineales en los ejes x,y,z; que mueven la plataforma en forma lineal hacia un lado y luego hacia el otro (acelerando y frenando para evitar el golpe brusco).
 - Un script para un movimiento circular de la plataforma, que se traslada circularmente alrededor de un punto fijo en el espacio.
 - Un script que administra el color de la plataforma (modificando su material). La plataforma tiene dos materiales: uno "transparente" (en la práctica no se ha hecho transparente, sino de un color parecido al cielo para confundirlo) y otro rojo. La plataforma se hace visible cuando el jugador se sube a ella, y cuando levanta la vara hace visible durante 30 segundos a las dos plataformas adyacentes.

Cabe destacar que para cada plataforma se utiliza un solo script de movimiento, y los otros tres se deshabilitan, a fin de que la plataforma tenga un solo movimiento previsible. La escena tiene muchas plataformas móviles, cada una empieza junto al extremo de movimiento de otra, de modo que el jugador puede recorrer la ciudad desde plataformas que vuelan.

- Asistente de la ciudad

En la escena hay un asistente parado sobre una plataforma móvil, el cual realiza en loop un movimiento ascendente de la vara para mostrarle al usuario cómo ver el camino.



Imagen 14: Asistente de la ciudad al inicio de la sesión

- Interacción del usuario

Cuando el usuario levanta la vara, llega el código numérico "6" desde el servidor WebSocket levantado en el ESP-8266. El camino del mensaje es el siguiente: el objeto WebSocket recibe el mensaje, si es un 6 lo transmite al objeto Gestor-Pisos, que a su vez se lo envía a AvatarPersonaje. Finalmente este llama al método revelarCamino del script VisualizarPlataforma de la plataforma de la que es hijo, en caso de estar subido a una. Cada plataforma tiene referencias a sus plataformas vecinas para llamar al método aparecerTemporalmente(), que cambia su material durante 30 segundos.

5.1.4 Isla Virtual

- Entorno y objetos de la escena Este entorno también se ha descargado del assetstore de Unity. Se trata de una isla junto a una ruta, donde hay vegetación, casas, automóviles y algunos otros elementos.



Imagen 15: Vista de la isla virtual

La idea de este escenario es que el usuario habilite un recorrido dibujando una figura geométrica. Para esto, se han añadido dos objetos a la escena:

- Automóvil volador
Como automóvil volador se reutilizó un prefab disponible en el entorno de la ciudad. A este objeto se le asignó el tag "MovingPlatform" a fin de que el usuario se mueva con el automóvil cuando este comience el recorrido.
- Tablero de figuras
El tablero de figuras tiene un círculo, un triángulo y un cuadrado en relieve. Para que el automóvil "reaccione" a la figura generada existen los scripts DibujarCuadrado, DibujarTriangulo y DibujarCirculo, que detectan la colisión de la vara con pequeños cubos 3D transparentes, ubicados en el contorno de la figura a dibujar. Los scripts funcionan de la siguiente forma: cuando en un rango de tiempo no mayor a cinco segundos la vara ha colisionado con todos los cubos 3D de la figura, entonces se dispara una variable tipo trigger del componente Animator del auto volador. Hay tres variables trigger, cada una dispara una animación predefinida, que consiste en una traslación (con rotación) del auto volador a través del escenario virtual.

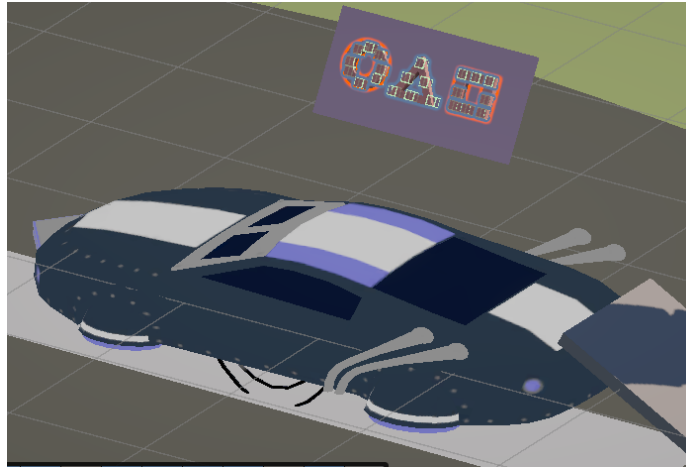


Imagen 16: Vista del auto volador y tablero de figuras

- **Interacción del usuario** En este caso, se utiliza la posición de la vara en la realidad virtual, proveniente del hand-tracking, por lo que no requiere datos provenientes de la vara real enviados a través del WebSocket. La vara es hija de la mano derecha en la jerarquía, por lo que se moverá cuanto mueva la mano el jugador. La detección de la colisión de un punto con la vara requiere que la vara tenga el box-collider y un tag "Vara". Luego, los cubos 3D que se usan como puntos de detección tienen un pequeño script llamado `DetectorVara` que usa raycast para determinar si hay otro objeto cerca, y si ese objeto tiene el tag "Vara".

5.1.5 Configuración de la vara

Para que la vara real funcione, es necesario pre-entrenar el módulo de reconocimiento de voz, esto se puede hacer con los ejemplos que vienen en la librería de `VoiceRecognition`. De esta manera, el módulo reconocerá las palabras correspondientes a los cinco animales con los que se juega en este entorno. Además, es necesario leer los valores del mpu y determinar si la vara se está agitando o elevando; y también montar el WebSocket server para enviar datos cuando se detecte que el usuario intentó crear un animal (números del 1 al 5) o en caso de que la vara esté acelerando hacia arriba, se asume que el usuario la está elevando, por lo que se envía un 6. El programa `varaServer.ino` (disponible en el repositorio) realiza la siguiente funcionalidad:

- En la función `setup` se configura el MPU y se inicializa el `VoiceReconizer`. Luego se monta el `WebSocketServer` en una IP fija en un puerto asignado: se enviará información cuando ocurra algo que deba ser comunicado, no hay envío periodizado.
- En la función `loop`, se procesa la información de la siguiente manera:

- Se lee periódicamente el MPU, obteniendo la ternas de valores (ax, ay, az) y (gx, gy, gz) que corresponden a aceleraciones y velocidades de giro.
- Previamente, se realizaron una serie de pruebas con el sensor en las cuales se determinaron los offsets del giroscopio y su sensibilidad al movimiento, dada por la siguiente curva (la curva corresponde a $\sqrt{(gx^2 + gy^2 + gz^2)}$). El pico corresponde a la máxima intensidad de movimiento, mientras que los valles corresponden a los momentos en que estaba quieta la vara. De esta manera, se establece un umbral a partir del cual se considera que el usuario está agitando la vara, lo que se refleja en una variable booleana.



Imagen 17: Valores experimentales del movimiento de la vara

- En el caso de la aceleración, la suma algebraica de las tres aceleraciones (suma algebraica $ax + ay + az$, no su composición vectorial) tiene un valor negativo siempre que la vara está quieta, ya que el acelerómetro en lugar de dar cero da la aceleración de la gravedad. Cuando esta composición está por encima de cierto umbral de referencia (fijado experimentalmente) se considera que la vara se está moviendo hacia arriba, compensando la aceleración de la gravedad, lo que se refleja en otra variable booleana que indica la acción de levantar la vara.
- Periódicamente se consulta al Voice Recognizer si ha detectado una palabra de las que tiene entrenada. En caso de ser así, devuelve un número que va de 1 a 5 según que animal haya dicho el jugador. Este valor se guarda en una variable.
- El WebSocket tiene habilitado un evento que hace que cualquier cliente conectado pueda interrumpir la ejecución enviando un número. El cliente que puede realizar esta interrupción es el intérprete humano de voz, el cual puede puentear al voice recognizer, guardando en una variable un número del 1 al 5 del animal que el jugador haya dicho.

Esto es útil (más bien necesario) porque el módulo de reconocimiento de voz no es sensible al cambio de timbre y tono de voz respecto a aquel con el que fue entrenado, por lo que muchas veces no reconoce la palabra dicha por el usuario. En esa situación, el operador, o asistente en la terapia del niño puede pulsar el botón de la palabra que ha dicho.

- Cuando se dan simultáneamente estas dos condiciones: el usuario ha dicho una palabra válida del juego y la variable que indica que la vara se está agitando, se manda por el WebSocket el número correspondiente al animal mencionado, para que aparezca en el juego. Si la vara no se agita, el mensaje no se manda.
- Cuando se da la condición de jugador elevando la vara, se envía un 6 por el web socket. También se permite que este número sea enviado por un cliente del Web Socket (en la práctica, solo se usó para testear la comunicación).

5.1.6 Intérprete Humano de Voz

En este caso, es posible abrir un fichero `< html >` que ejecuta una función en el navegador para escuchar el puerto correspondiente del Web Socket. Este enfoque es útil porque es una solución sencilla, que permite trabajar como cliente desde cualquier plataforma. No obstante, solo sirve para comunicarse como cliente, pero no para montar el servidor. Este fichero desarrolla seis botones sencillos, con los cuales se permite enviar números del 1 al 6 correspondientes a los cinco animales de la granja y a la elevación de la vara. El fichero se encuentra disponible en el repositorio, su nombre es **interprete de voz.html**



Imagen 18: Vista del fichero en ejecución

5.2 Versión Simplificada

La versión simplificada implica la no utilización de la vara física, y en su lugar los mandos. En esta situación, no se dispone de reconocimiento de voz, por

lo que se montará el servidor en una aplicación de PC y se realizan algunos pequeños cambios en el entorno.

5.2.1 Interprete Humano de Voz

Para poder montar el server, se puede programar en python utilizando los módulos websocket y tkinter (nativo de python) para la GUI. El programa corre la ventana en el hilo principal, y el websocket en el hilo secundario. El programa completo tiene 265 líneas de código, pero es bastante sencillo, su funcionalidad es enlazar el botón que se presenta al usuario con el código numérico que envía el WebSocket Server.

Para este caso se han agregado dos códigos numéricos: el 7 cerrar corral y el 8 abrir corral. En la siguiente sección se explicará que hacen estos dos códigos.

Cabe destacar que el servidor se monta en la red WiFi donde está conectada la PC desde donde se ejecuta la aplicación, pero en este caso no se necesita tener una IP fija en la computadora, sino que el WebSocket puede montarse sobre otra IP, que será escuchada por los clientes.

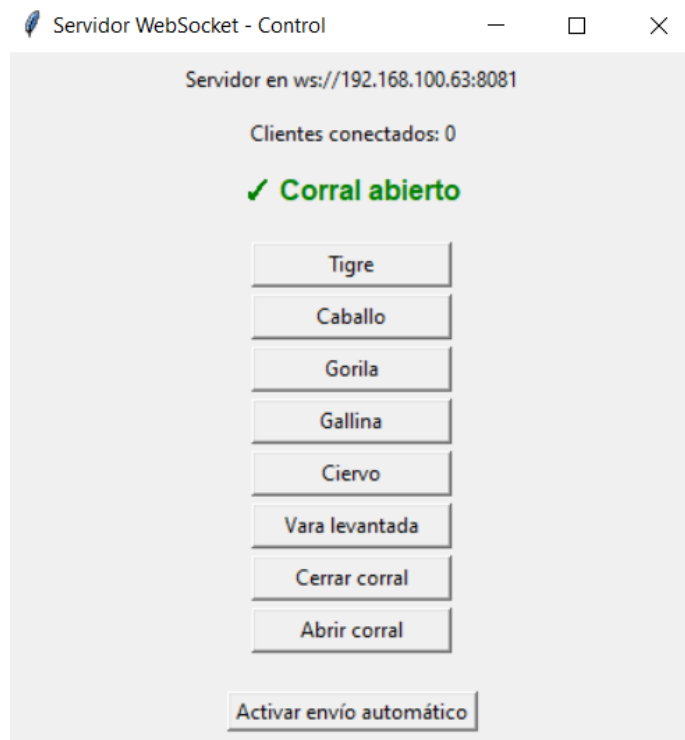


Imagen 19: Vista del Web Socket Server en ejecución

5.2.2 Modificaciones en el entorno de realidad virtual

Las modificaciones realizadas son pocas, se detallan a continuación:

- Agitación de la vara: Para detectar la agitación de la vara ya no se usa el MPU, ni tampoco se toma el movimiento de la vara en la realidad virtual, sino que se toma como único criterio el mensaje enviado por el intérprete humano de voz, que en este caso es el servidor programado en python. Se considera que el criterio del operador es suficiente, ya que debe pulsar el botón correspondiente cuando el niño dice la palabra y agita la vara.
- Elevación de la vara: En este caso, se modifica el script GestorPisos para que éste evalúe la posición de la vara en la realidad virtual: cuando esté por encima de la cabeza del avatar se llamará al método revelarCamino() del Avatar. En la versión completa, este método se llamaba cuando se recibía un 6 por el WebSocket, mientras que en este caso es posible llamarlo de ambas maneras. En este punto es pertinente destacar que, si bien el programa puede realizar hand-tracking, es recomendable el uso de los mandos. ¿Por qué? Eventualmente, el jugador puede elevar su mano por encima de la cabeza con las cámaras del cabezal apuntando hacia abajo, en cuyo caso no detectará la posición real de las manos y el programa "perderá" la acción realizada por el usuario. Sin embargo, si el jugador tiene tomados los mandos, al elevar la mano derecha el Meta Quest detectará este movimiento, y podrá realizar la acción correspondiente (revelar camino).
- Corral transparente en el auto volador: Después de realizar varios testeos de funcionamiento, se vio que en ciertos recorridos a bordo del auto volador en la escena de la isla virtual, según la posición en la que haya quedado el jugador, puede "resbalarse" y caer del auto. Esto puede repercutir negativamente sobre la experiencia del usuario, por lo que se agregó un corral transparente (hecho con cubos 3D) alrededor del auto volador, que funciona como un mecanismo de seguridad para no caerse del auto. Este corral está al principio inhabilitado (ya que si no lo estuviera, el jugador tendría dificultades para subir a bordo del auto) y se maneja desde la aplicación del WebSocket server, con los botones "cerrar corral" y "abrir corral".

6 RECURSOS UTILIZADOS Y ACCESO AL PROYECTO

- Para programar el ESP8266 se utilizó IDE Arduino 1.8.19, pero el código es compatible con cualquier versión que pueda compilar para tarjetas Node MCU.
- El entorno fue desarrollado en la versión 6000.3.2f1 de Unity Editor.
- El aplicativo intérprete de voz se desarrolló como `!htmlj` en un editor de texto plano para la versión completa. El programa para la versión simplificada fue desarrollado en VSCode con python.
- Se utilizó Audacity para editar, mezclar y normalizar el audio utilizado. Los efectos sonoros se tomaron de pixabay y la voz fue generada por IA utilizando Speaktor.
<https://pixabay.com/es/sound-effects/search/juego/>
<https://speaktor.com/>
- El avatar del personaje, así como los asistentes y sus animaciones, fueron descargados como characters de Mixamo:
<https://www.mixamo.com//>
- Desde assetstore en Unity se descargaron los paquetes para las varas, los portales y los escenarios virtuales.
<https://assetstore.unity.com/3d>
- La librería NativeWebSocket se descargó del siguiente repositorio:
<https://github.com/andel/NativeWebSocket>
- Para el aprendizaje inicial de Unity, enfocado a los elementos usados en este proyecto se recomiendan los canales @ibiza360vr y @jucarave:
<https://www.youtube.com/@Ibiza360VR>
<https://www.youtube.com/watch?v=PesrC6HcYB8>
- Para implementar un WebSocketServer en el ESP8266, se recomienda el siguiente tutorial:
<https://www.youtube.com/watch?v=lcJzVP20McM>
- En el siguiente repositorio, puede accederse al proyecto completo:
<https://github.com/albamar92/Realidad-Virtual/>
- En los siguientes enlaces puede accederse a material audiovisual de las experiencias de uso.
 - Inicio del entorno:
<https://youtu.be/NlkbRkZNe3E>

- Ingreso en la Granja Virtual:
<https://youtu.be/kBov3M1qNeA>
- Ingreso en la Ciudad Virtual:
<https://youtu.be/t4CF2uMHFxs>
- Ingreso en la Isla Virtual:
<https://youtu.be/WFrp0otZw9k>
- En los siguientes enlaces, puede accederse a material audiovisual explicativo del proyecto.
 - Proyecto en Unity: versión completa
 - * Inicio: <https://youtu.be/Zq7RLBrpBs>
 - * Granja: <https://youtu.be/ViLj4MkZ3Vs>
 - * Ciudad: <https://youtu.be/pdOsYQhMnt8>
 - * Isla: <https://youtu.be/4Dl0E3wNgSA>
 - Proyecto en Unity: versión simplificada
<https://youtu.be/hlD4FFUYnw>
 - Software y Hardware de la vara
<https://youtu.be/LCp961rw64>
 - Intérprete de voz (cliente y servidor)
<https://youtu.be/n2M4NTBls-k>

7 CONCLUSIONES

1. Se logró consolidar los conceptos y perspectivas vistos en el cursado de la materia en un proyecto realista.
2. Se consiguió un prototipo funcional, aunque es susceptible a muchas mejoras y agregados
3. Se aprendió mucho respecto al uso de herramientas de realidad virtual, cuya utilidad es genérica y puede servir para una gran diversidad de aplicaciones. En esta misma línea, fue muy útil y valioso trabajar con un hardware específicamente pensado para esto, como Meta Quest.
4. El tiempo de aprendizaje para este tipo de herramientas es largo, aunque hay mucha documentación disponible y es posible aprender de forma sistemática.
5. El desarrollo de este tipo de aplicaciones es bastante lento, en comparación con otras áreas tecnológicas que son más ágiles. También se requiere de gran capacidad de cómputo.
6. El hardware utilizado para la vara es susceptible de ser mejorado y complementado. Originalmente se pensó en trabajar con magnetómetros y módulos de vibración para realimentación háptica en la vara, lo cual se simplificó por razones económicas y de disponibilidad de componentes. No obstante, con una inversión relativamente baja podría mejorarse mucho.
7. Se intentó trabajar el reconocimiento de audio usando Speech to Text de Meta Quest, sin embargo no se logró satisfactoriamente debido a que ciertas herramientas parecen discontinuadas o no disponibles en español. Sería interesante estudiar este aspecto en mayor profundidad para buscar una solución más sólida al reconocimiento de voz.
8. Dada la temática elegida para el proyecto, en estos mismos escenarios hay una enorme posibilidad de introducir variantes, nuevas acciones, objetos para que el niño descubra en el juego, etc. Para acotarlo a los objetivos del espacio curricular, se priorizó diversificar y abarcar estos tres escenarios, sin darle una gran profundidad al desarrollo de cada uno, lo cual llevaría mucho tiempo.
9. En base a lo aprendido, se concluye que el trabajo con colisiones es de los aspectos más complejos a la hora de desarrollar proyectos.